
Technical University of Crete
School of Electrical and Computer Engineering

Course: Optimization

Instructor: Athanasios P. Liavas

Student: Nikolaos Malamas (2020030180)

Exercise 3 (150/1000)

Report Delivery Date: **December 8, 2025**

In the first part of this exercise, we shall solve an unconstrained convex optimization problem using the gradient descent and the Newton methods.

(70) Let $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{A} \in \mathbb{R}^{m \times n}$, with rows $\mathbf{a}_i^T$, for $i = 1, \dots, m$. Consider the function $f : \text{dom} f \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$, defined as

$$f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} - \sum_{i=1}^m \log(b_i - \mathbf{a}_i^T \mathbf{x}) = \mathbf{c}^T \mathbf{x} - \text{sum}(\log(\mathbf{b} - \mathbf{A}\mathbf{x})), \quad (1)$$

with $\mathbf{x} \in \mathbb{R}^n$ and $m > n$ (or $m \gg n$). For example, $(n, m) = (2, 20)$, $(50, 200)$. If you have patience and enough memory in your computer, you may set $(n, m) = (300, 800)$ or even larger values.

Some observations are as follows (see [Boyd–Vandenberghe, pages 141, 419–422, 458–459, 472, 492](#)):

- The set $\text{dom} f$ contains only the points $x \in \mathbb{R}^n$ for which the arguments of the logarithms are positive.

Prove that:

1. (5) The set $\text{dom} f$ is convex.

Answer:

Let us assume that $x_1, x_2 \in \text{dom} f$. Then

$$\mathbf{A}\mathbf{x}_1 < \mathbf{b}$$

$$\mathbf{A}\mathbf{x}_2 < \mathbf{b}$$

Then, for any $\theta \in [0, 1]$:

$$\left. \begin{array}{l} \theta \mathbf{A} \mathbf{x}_1 < \theta \mathbf{b} \\ (1 - \theta) \mathbf{A} \mathbf{x}_2 < (1 - \theta) \mathbf{b} \end{array} \right\} \stackrel{(+)}{\Rightarrow} \theta \mathbf{A} \mathbf{x}_1 + (1 - \theta) \mathbf{A} \mathbf{x}_2 < \theta \mathbf{b} + (1 - \theta) \mathbf{b}$$

$$\Rightarrow \mathbf{A}(\theta \mathbf{x}_1) + \mathbf{A}(1 - \theta) \mathbf{x}_2 < \mathbf{b}$$

$$\Rightarrow \mathbf{A}[\theta \mathbf{x}_1 + (1 - \theta) \mathbf{x}_2] < \mathbf{b}$$

$$\Rightarrow \mathbf{A} \hat{\mathbf{x}} < \mathbf{b} \Rightarrow \hat{\mathbf{x}} \in \mathbf{dom} f$$

Thus any convex combination of the points x_1, x_2 is an element of $\mathbf{dom} f$. Therefore, $\mathbf{dom} f$ is a **convex** set.

2. (5) Function f is convex.

Answer:

We will derive $\nabla^2 f$ thinking that it may satisfy the second order condition for convexity.

From Boyd's book we get that the gradient is

$$\nabla f(\mathbf{x}) = \mathbf{c} - \sum_{i=1}^m \frac{1}{b_i - \mathbf{a}_i^T \mathbf{x}} \mathbf{a}_i$$

By the linearity of the derivative and the product rule for the “functions” $\frac{1}{b_i - \mathbf{a}_i^T}$ and $\mathbf{a}_i$ we can compute the hessian as

$$\nabla^2 f(\mathbf{x}) = \sum_{i=1}^m \frac{1}{(b_i - \mathbf{a}_i^T)^2} \mathbf{a}_i \mathbf{a}_i^T$$

If we set $\mathbf{a}_i \mathbf{a}_i^T := \mathbf{M}_i \in \mathbb{R}^{n \times n}$ then each of those matrices would be positive definite, because for any non-zero vector $\mathbf{z} \in \mathbb{R}^n$

$$\mathbf{z}^T \mathbf{M}_i \mathbf{z} = \mathbf{z}^T \mathbf{a}_i \mathbf{a}_i^T \mathbf{z} = (\mathbf{a}_i^T \mathbf{z})^T \underbrace{\mathbf{a}_i^T \mathbf{z}}_{\mathbf{v}} = \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|^2 = \|\mathbf{a}_i^T \mathbf{z}\|^2 > 0$$

Thus, since the quantity $\frac{1}{(b_i - \mathbf{a}_i^T)^2}$ is (strictly) positive, $\nabla^2 f(\mathbf{x}) \succ \mathbf{O}$

Therefore, the second order condition for convexity holds true, making function f **convex**.

- Observe that if $b_i > 0$, for $i = 1, \dots, m$, then $\text{dom} f \neq \emptyset$, because $\mathbf{x} = \mathbf{0}$ is a feasible point (in the experiments, it makes sense to *always* use this convention).
- Function f may be unbounded from below. In this case, the problem has no solution (no need to do something for it - `cvx` will identify these cases).
- If a solution $\mathbf{x}_*$ exists, then it lies in the interior of $\text{dom} f$, because when we approach the boundary of $\text{dom} f$ the value of the function increases without bound. This means that a necessary and sufficient condition for $\mathbf{x}_*$ to be an optimal point is $\nabla f(\mathbf{x}_*) = \mathbf{0}$.

Our study will proceed as follows.

1. Create quantities $\mathbf{A}$, $\mathbf{b}$, and $\mathbf{c}$ randomly (in order to be certain that the problem is feasible, create $\mathbf{b}$ with non-negative elements).

Answer:

```
% 1. Create A, b, and c randomly
A = randn(m,n);
b = rand(m,1);
c = randn(n,1);
```

2. Minimize f using `cvx`. If the problem has a solution, `cvx` will compute it, otherwise it will display a message saying that the problem has no solution.

Answer:

```
% 2. Minimize f using cvx
cvx_begin
    variables x_cvx(n)
    minimize(c' * x_cvx - sum(log(b - A * x_cvx)))
cvx_end

fprintf('Optimal point as given by CVX (x_cvx):(%.4f, %.4f)\n',
    , x_cvx(1), x_cvx(2));
fprintf('\n');
```

3. (10) If $n = 2$, then plot f and its level sets in the neighborhood of the optimum point. You can check whether a point $\mathbf{x}$ belongs to the domain of f by checking the argument of the logarithm at this point. Otherwise you can give an arbitrarily large value to f at this point (for example, $f(\mathbf{x}) = 10^3$).

Answer :

We plot f both with and without visible barriers

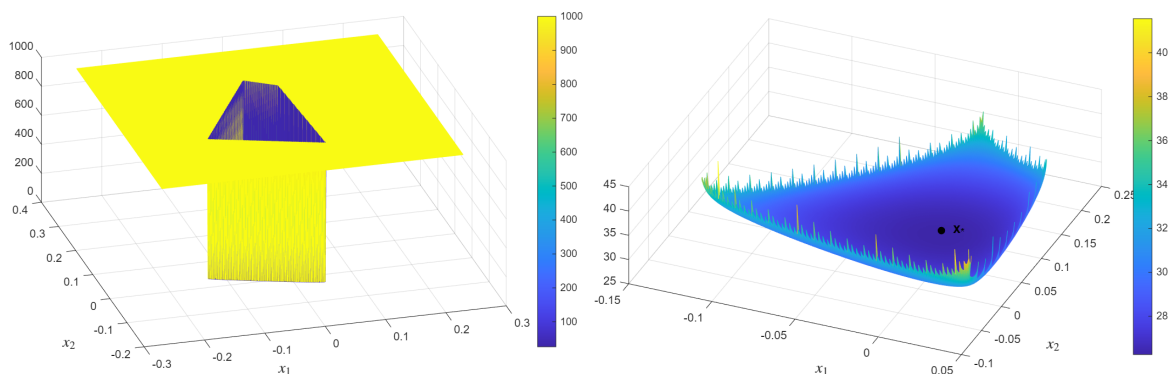


Figure 1: mesh plots of $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} - \sum_{i=1}^m \log(b_i - \mathbf{a}_i^T \mathbf{x})$ with visible barriers (left) and without (right)

Also we plot the level sets of f

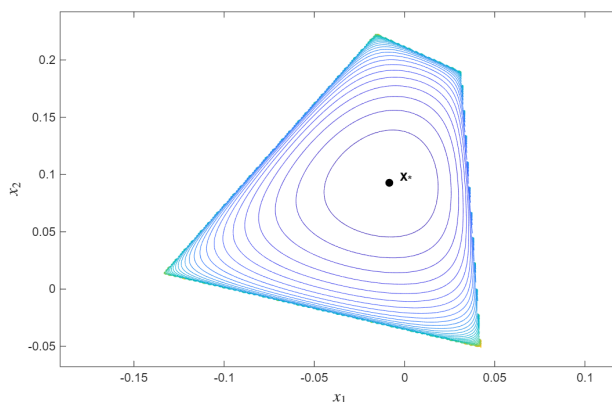


Figure 2: Level sets of $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} - \sum_{i=1}^m \log(b_i - \mathbf{a}_i^T \mathbf{x})$

The code that yielded the results above:

```
% 3. Plot f and its level sets near the optimum point
if (n == 2)
    step = 10^(-3); % 10^-4 takes up to 3 minutes to run
    x1 = x_cvx(1) - 0.25: step: x_cvx(1) + 0.25 - step;
    x2 = x_cvx(2) - 0.25: step: x_cvx(2) + 0.25 - step;
    [X1,X2] = meshgrid(x1, x2);

    % Calculate ALL the values of f,
    % even in points outside the domain
    for i = 1:length(X1(:,1))
        for j = 1:length(X1(:,1))
            x_tmp = [X1(i,j); X2(i,j)];
            [X1_(i,j), X2_(i,j), f_(i,j)] = f(A, b, c, x_tmp);
        end
    end

    % Full version of f
    f_log_bar = f_;

    % Exclude the points that do NOT belong in dom{f}
    invalid = (X1_ == -1);
    f_(invalid) = NaN;

    % Plot f
    figure
    mesh(X1_,X2_,f_, 'FaceColor', 'interp');
    hold on;
    [~,~,z0] = f(A,b,c,x_cvx);
    plot3(x_cvx(1), x_cvx(2), z0+1, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 6);
    text(x_cvx(1) + 0.005, x_cvx(2) + 0.005, z0 + 1, 'x_*', 'FontSize', 12, 'Color', 'k', 'FontWeight', 'bold');
    colorbar;
    xlabel('$x_1$', 'FontSize', 14, 'Interpreter', 'latex');
    ylabel('$x_2$', 'FontSize', 14, 'Interpreter', 'latex');
    hold off;
```

```

% Plot f with visible barriers
figure
mesh(X1,X2,f_log_bar,'FaceColor','interp');
hold on;
[~,~,z0] = f(A,b,c,x_cvx);
plot3(x_cvx(1), x_cvx(2), z0, 'ko', 'MarkerFaceColor', 'k',
      'MarkerSize', 6);
line ([x_cvx(1) x_cvx(1)], [x_cvx(2) x_cvx(2)], [z0 0], '
      Color', 'r', 'LineWidth', 1.5, 'LineStyle', ':');
text(x_cvx(1) + 0.005, x_cvx(2) + 0.005, z0 + 1, 'x_*', '
      FontSize', 12, 'Color', 'k', 'FontWeight', 'bold');
colorbar;
xlabel('$x_1$', 'FontSize', 14, 'Interpreter', 'latex');
ylabel('$x_2$', 'FontSize', 14, 'Interpreter', 'latex');
hold off;

minf = min(f_(:));
maxf = max(f_(:));

ls = linspace(minf, maxf, 40); % 40 evenly spaced lines

% Plot the level sets of f
figure
contour(X1,X2,f_,ls);%,'ShowText','on');
hold on;
plot(x_cvx(1), x_cvx(2), 'ko', 'MarkerFaceColor', 'k', '
      MarkerSize', 6);
hold on;
text(x_cvx(1) + 0.005, x_cvx(2) + 0.005, 'x_*', 'FontSize',
      12, 'Color', 'k', 'FontWeight', 'bold');
xlabel('$x_1$', 'FontSize', 14, 'Interpreter', 'latex');
ylabel('$x_2$', 'FontSize', 14, 'Interpreter', 'latex');
% axis([min(0, x_cvx(1)) - 0.2 max(0, x_cvx(1)) + 0.2 min
      (0, x_cvx(2)) - 0.2 max(0, x_cvx(2)) + 0.2])
hold off;
end

```

4. (20) Assuming that $\mathbf{x} = \mathbf{0}$ is a feasible point, minimize f using the gradient algorithm with backtracking line search, starting from $\mathbf{x}_0 = \mathbf{0}$. Since $\text{dom} f \subset \mathbb{R}^n$, the implementation will have the following main difference from the baseline implementation of the gradient descent algorithm.

- In step $(k + 1)$, given $\mathbf{x}_k$ and $\nabla f(\mathbf{x}_k)$, and having set $t = 1$, before starting the backtracking line search, you should check whether the point $\mathbf{x}_k + t\Delta\mathbf{x}_k$ belongs to $\text{dom} f$ or not. If it does not belong to $\text{dom} f$, then you must put $t := \beta t$ (β is the backtracking parameter) and repeat this process until you find a point that belongs to $\text{dom} f$ ¹. When you arrive at a point that belongs to $\text{dom} f$, then you can proceed to the typical backtracking line search.

Answer:

We implement the Gradient Descent algorithm with the appropriate check on whether the point we are about to traverse to is in $\text{dom} f$ or not:

```
function [xk,f_xk] = Backtracking_LS_log_bar(A, b, c, x0,
    epsilon, alpha, beta, x_opt)

    x = x0;
    xk = [x];

    [~,~,fun_val] = f(A, b, c, x0);
    f_xk = [fun_val];
    grad = g(A, b, c, x0);

    iters = 0;

    while (norm(x - x_opt) > epsilon)
        iters = iters + 1;
        fprintf('Iteration: %d\n', iters);
        t = 1;

        % Iteratively search for a point in the domain of f
```

¹It may help you in the understanding of the behavior of the method if you print a message every time point $\mathbf{x} + t\Delta\mathbf{x}$ does not belong in $\text{dom} f$. You may observe different behavior far away and close to the solution.

```

while ( any(b - A*(x - t*grad) <= 0) )
    x_tmp = x - t*grad;
    fprintf('x_{k+1} = [%.4f,%.4f] does NOT belong to
           dom{f} \n', x_tmp(1), x_tmp(2));

    t = beta * t;
end

% After finding a valid point, proceed with the
% standard BT-LS
[~,~,f0] = f(A, b, c, x - t*grad);
while (fun_val - f0 < alpha*t*norm(grad)^2)
    t = beta * t;
    [~,~,f0] = f(A, b, c, x - t*grad);
end

fprintf('-----\n');

% Update
x = x - t*grad;
xk = [xk x];
 [~,~,fun_val] = f(A, b, c, x);
f_xk = [f_xk fun_val];
grad = g(A, b, c, x);

end

end

```

which is used in the main script as follows

```

% 4. Perform GD w/BT-LS and x0 = 0
x0 = zeros(n,1);

alpha = 0.1;
beta = 0.7;
epsilon = 10^(-6);

fprintf('Now executing the gradient descent algorithm...\n');

```



```
[X_bt, F_bt] = Backtracking_LS_log_bar(A, b, c, x0, epsilon,
    alpha, beta, x_cvx);

if (round(F_bt(end),4) == round(cvx_optval,4))
    fprintf('GD with BT-LS found the same solution as cvx!\n\n
    ');
end

fprintf('-----\n');
```

After executing the preceding code, MatLab command window prints

Command Window

GD with BT-LS found the same solution as cvx!

5. (20) Following the analogous procedure, minimize f using the Newton algorithm.

Answer:

Same applies for the implementation of the Newton algorithm

```
function [xk,f_xk] = Newton_log_bar(A, b, c, x0, epsilon,
    alpha, beta)

    x = x0;
    xk = [x];

    [~,~,fun_val] = f(A, b, c, x0);
    f_xk = [fun_val];
    grad = g(A, b, c, x0);
    hess = h(A,b,x0);

    iters = 0;

    while (1)
        iters = iters + 1;
        fprintf('Iteration: %d\n', iters);

        Dx_Nt = -inv(hess) * grad;
        lambda_sq = grad.' * inv(hess) * grad;
```

```

if (lambda_sq/2 <= epsilon)
    break;
end

t = 1;
% Iteratively search for a point in the domain of f
while ( any(b - A*(x + t*Dx_Nt) <= 0) )
    x_tmp = x + t*Dx_Nt;
    fprintf('x_{k+1} = [%.4f,%.4f] does NOT belong to
            dom{f} \n', x_tmp(1), x_tmp(2));

    t = beta * t;
end

fprintf('-----\n');

% After finding a valid point, proceed with the
    standard BT-LS
[~,~,f0] = f(A, b, c, x + t*Dx_Nt);
while (f0 > fun_val + alpha*t*norm(grad)^2)
    t = beta * t;
    [~,~,f0] = f(A, b, c, x + t*Dx_Nt);
end

% Update
x = x + t*Dx_Nt;
xk = [xk x];
[~,~,fun_val] = f(A, b, c, x);
f_xk = [f_xk fun_val];
grad = g(A, b, c, x);
hess = h(A,b,x);

end

end

```

which is used as follows in the main script

```

% 5. Minimize f using the Newton algorithm
x0 = zeros(n,1);

alpha = 0.1;
beta = 0.7;
epsilon = 10^(-6);

fprintf('Now executing the Newton algorithm...\n');
[X_Nt, F_Nt] = Newton_log_bar(A, b, c, x0, epsilon, alpha,
    beta);

if (round(F_Nt(end),4) == round(cvx_optval,4))
    fprintf('Newton method found the same solution as cvx!\n\n
        ');
end

fprintf('-----\n');

```

After executing the preceding code, MatLab command window prints

Command Window

Newton method found the same solution as cvx!

6. (10) Plot, using `semilogy` ($f_{\text{gradient}}(\mathbf{x}_k) - p_*$) and ($f_{\text{newton}}(\mathbf{x}_k) - p_*$) as a function of step k . What do you observe for small and large values of the pair (n, m) ?

Answer:

By monitoring the difference between $f(\mathbf{x}_k)$ and $f(\mathbf{x}_*)$, we are able to plot each one of them using `semilogy`

```

% 6. Plot (f_GD - p_*) and (f_Nt - p_*) vs k
figure
semilogy([1:length(F_Nt)]', (F_Nt-F_Nt(end)), 'b', 'LineWidth',
    1.5);
hold on;
semilogy([1:length(F_bt)]', (F_bt-F_bt(end)), 'r', 'LineWidth',
    1.5);

```

```

grid on;
xlabel('$k$', 'FontSize', 13, 'Interpreter','latex');
ylabel('$\log(f(\mathbf{x}_k) - p_*)$', 'FontSize', 13, 'Interpreter', 'latex');
legend('Newton', 'Backtracking LS');
axis tight

```

By running the preceding code we get the following plots

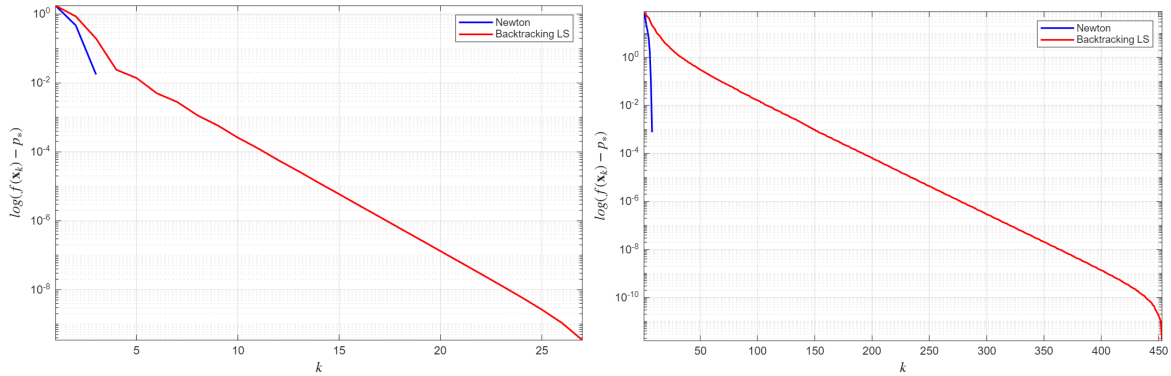


Figure 3: **semilogy** of $(f_{\text{gradient}}(\mathbf{x}_k) - p_*)$ and $(f_{\text{newton}}(\mathbf{x}_k) - p_*)$ vs k , for $n = 2$, $m = 20$ (left) and $n = 50$, $m = 200$ (right)

As we can see, in both cases of the pair (n, m) the convergence of the Newton method is significantly faster than the one of the GD with BT-LS. In fact, the Newton method shows quadratic convergence, which is not really visible because we use **semilogy** to plot, while the GD algorithm, shows a linear one, as we have seen in previous exercises.

The quadratic convergence of the Newton algorithm is to be expected since, as we know from our theory, when starting our descent from a point sufficiently close to the solution (here $\mathbf{0}$), then the number of digits in the solution provided by the Newton algorithm doubles on each iteration.

In the second part of this exercise, we shall solve simple (but important) optimization problems by explicitly solving the KKT conditions. In all cases, start with a drawing of the problem.

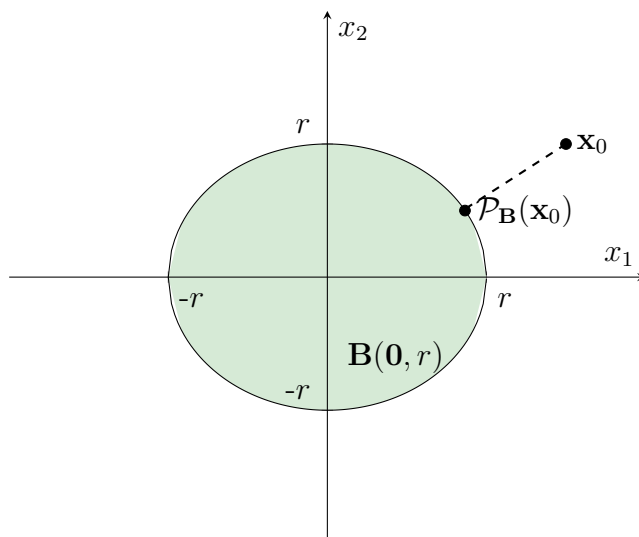
(80) The problems are the following:

1. (20) Compute the projection of $\mathbf{x}_0 \in \mathbb{R}^n$ onto $\mathbf{B}(\mathbf{0}, r) := \{\mathbf{x} \in \mathbb{R}^n \mid \|\mathbf{x}\|_2 \leq r\}$.

(a) Draw a scheme of the problem.

Answer:

We consider the case where $n = 2$, because it is easier to draw; all the results can be generalized for any $n \in \mathbb{R}$.



- (b) Write down the optimization problem you must solve, in terms of **differentiable** functions.

Answer:

The problem we want to solve

$$\begin{aligned} \min_{\mathbf{x}} f_0 &= \|\mathbf{x}_0 - \mathbf{x}\|^2 \\ \text{subject to } &\|\mathbf{x}\| - r \leq 0 \end{aligned}$$

but since the constraint of the first problem is a *nondifferentiable* function, we will solve an equivalent problem with differentiable constraints. The equivalent problem from which we will derive the solution of the initial is as follows:

$$\begin{aligned} \min_{\mathbf{x}} f_0 &= \|\mathbf{x}_0 - \mathbf{x}\|^2 \\ \text{subject to } f_1(\mathbf{x}) &:= \|\mathbf{x}\|^2 - r^2 \leq 0 \end{aligned}$$

(c) Write the KKT conditions, in terms of the optimal parameters $(\mathbf{x}_*, \lambda_*)$.

Answer:

In order to write the KKT conditions we first calculate the gradients of the cost function and the inequality constraint function (no equality constraints h_i here)

$$\nabla f_0(\mathbf{x}) = 2(\mathbf{x} - \mathbf{x}_0) \text{ and } \nabla f_1(\mathbf{x}) = 2\mathbf{x}$$

Thus, the KKT conditions write

$$\begin{aligned} \begin{cases} \nabla f_0(\mathbf{x}) + \lambda \nabla f_1(\mathbf{x}) = 0 \\ \lambda \geq 0, f_1(\mathbf{x}) \leq 0 \\ \lambda f_1(\mathbf{x}) = 0 \end{cases} &\Rightarrow \begin{cases} 2(\mathbf{x} - \mathbf{x}_0) + \lambda 2\mathbf{x} = 0 \\ \lambda \geq 0, f_1(\mathbf{x}) \leq 0 \\ \lambda f_1(\mathbf{x}) = 0 \end{cases} \Rightarrow \begin{cases} (1 + \lambda)2\mathbf{x} = 2\mathbf{x}_0 \\ \lambda \geq 0, f_1(\mathbf{x}) \leq 0 \\ \lambda f_1(\mathbf{x}) = 0 \end{cases} \\ \Rightarrow \begin{cases} \mathbf{x}_* = \frac{\mathbf{x}_0}{\lambda + 1} \\ \lambda \geq 0, f_1(\mathbf{x}) \leq 0 \\ \lambda f_1(\mathbf{x}) = 0 \end{cases} \end{aligned}$$

(d) Consider the case $\lambda_* > 0$. What is the conclusion?

Answer:

By substituting the value of $\mathbf{x}$ from the first expression into the last one we get that

$$\begin{aligned} \lambda(\|\mathbf{x}\|^2 - r^2) = 0 &\Rightarrow \lambda\left(\frac{\|\mathbf{x}_0\|^2}{(\lambda+1)^2} - r^2\right) = 0 \stackrel{\lambda > 0}{\Rightarrow} \frac{\|\mathbf{x}_0\|^2}{(\lambda+1)^2} = r^2 \Rightarrow \frac{\|\mathbf{x}_0\|^2}{r^2} = (\lambda+1)^2 \Rightarrow \\ \lambda + 1 &= \pm \frac{\|\mathbf{x}_0\|}{r} \stackrel{\lambda > 0}{\Rightarrow} \lambda = \frac{\|\mathbf{x}_0\|}{r} - 1 \end{aligned}$$

which is surely positive ($\mathbf{x}_0$ is a point outside of the Euclidean ball $\mathbf{B}(\mathbf{0}, r)$) therefore it is an accepted solution.

By substituting the value of λ that we derived into the expression of the optimum point, we get

$$\mathbf{x}_* = \frac{\mathbf{x}_0}{\frac{\|\mathbf{x}_0\|}{r} - 1 + 1} = \frac{\mathbf{x}_0}{\|\mathbf{x}_0\|} r$$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$\boxed{(\mathbf{x}_*, \lambda_*) = \left(\frac{\mathbf{x}_0}{\|\mathbf{x}_0\|} r, \frac{\|\mathbf{x}_0\|}{r} - 1 \right)}$$

(e) Consider the case $\lambda_* = 0$. What is the conclusion?

Answer:

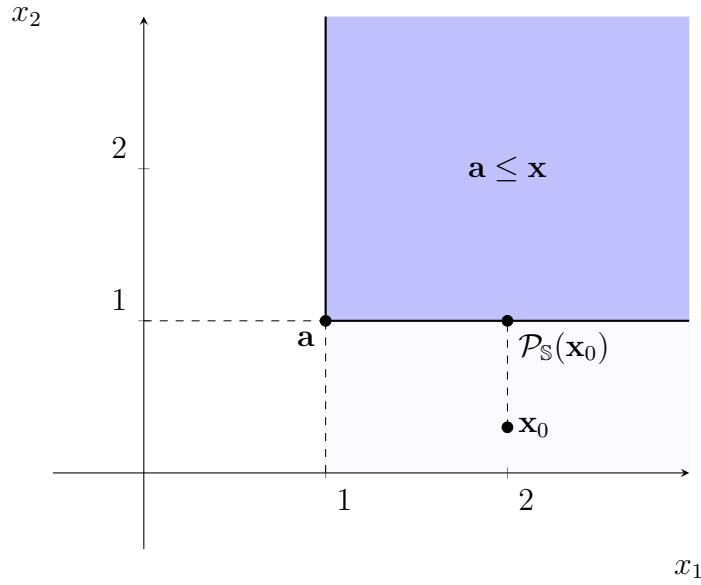
If $\lambda_* = 0$, then we can easily derive $\mathbf{x}_*$ to be $\mathbf{x}_0$ from the first relation of the KKTs. Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$\boxed{(\mathbf{x}_*, \lambda_*) = (\mathbf{x}_0, 0)}$$

2. (20) Let $\mathbf{a} \in \mathbb{R}^n$. Compute the projection of $\mathbf{x}_0 \in \mathbb{R}^n$ onto set $\mathbb{S} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a} \leq \mathbf{x}\}$.

Answer:

Again, we consider a 2D case (where $\mathbf{a} = (1, 1)$) for our drawing in order to get an intuition behind our problem



$\mathbb{S}$ is clearly convex since, for any $\theta \in [0, 1]$ and $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{S}$

$$\left. \begin{array}{l} \theta \mathbf{a} \leq \theta \mathbf{x}_1 \\ (1 - \theta) \mathbf{a} \leq (1 - \theta) \mathbf{x}_2 \end{array} \right\} \stackrel{(+)}{\Rightarrow} \theta \mathbf{a} + (1 - \theta) \mathbf{a} \leq \theta \mathbf{x}_1 + (1 - \theta) \mathbf{x}_2$$

$$\Rightarrow \mathbf{a} \leq \theta \mathbf{x}_1 + (1 - \theta) \mathbf{x}_2$$

$$\Rightarrow \mathbf{a} \leq \hat{\mathbf{x}} \Rightarrow \hat{\mathbf{x}} \in \mathbb{S}$$

Thus the problem we have to solve it of the type “Projection of a point onto a convex set”

$$\begin{aligned} \min_{\mathbf{x}} f_0 &= ||\mathbf{x}_0 - \mathbf{x}||^2 \\ \text{subject to } f_i(\mathbf{x}) &:= a_i - x_i \leq 0, i = 1, \dots, n \end{aligned}$$

The gradients are found to be

$$\nabla f_0(\mathbf{x}) = 2(\mathbf{x} - \mathbf{x}_0) \text{ and } \nabla f_i(\mathbf{x}) = -\mathbf{e}_i, \text{ where } \mathbf{e}_i := \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

Thus, the KKT conditions write

$$\begin{cases} \nabla f_0(\mathbf{x}) + \sum_{i=1}^n \lambda_i \nabla f_i(\mathbf{x}) = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_i f_i(\mathbf{x}) = 0 \end{cases} \Rightarrow \begin{cases} 2(\mathbf{x} - \mathbf{x}_0) - \sum_{i=1}^n \lambda_i \mathbf{e}_i = \mathbf{0} \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_i(a_i - x_i) = 0 \end{cases}$$

- If $\boldsymbol{\lambda} = \mathbf{0} \Rightarrow \lambda_i = 0$ then from the first equation of the KKTs

$$2(\mathbf{x} - \mathbf{x}_0) = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{x}_0$$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$\boxed{(\mathbf{x}_*, \boldsymbol{\lambda}_*) = (\mathbf{x}_0, \mathbf{0})}$$

- If $\boldsymbol{\lambda} > \mathbf{0}$ then, by dividing by λ_i in the last equation, we get

$$a_i - x_i = 0 \Rightarrow x_i = a_i$$

Subsequently, the first equation writes

$$\boldsymbol{\lambda} = 2(\mathbf{a} - \mathbf{x}_0)$$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

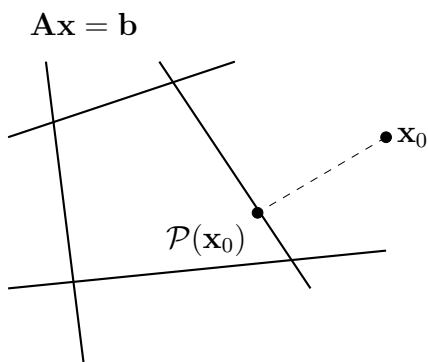
$$\boxed{(\mathbf{x}_*, \boldsymbol{\lambda}_*) = (\mathbf{a}, 2(\mathbf{a} - \mathbf{x}_0))}$$

3. (20) Let $\mathbf{A} \in \mathbb{R}^{p \times n}$, $\mathbf{b} \in \mathbb{R}^p$, and $\mathbf{x}_0 \in \mathbb{R}^n$. Solve the problem

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x}_0 - \mathbf{x}\|_2, \quad \text{subject to} \quad \mathbf{Ax} = \mathbf{b}. \quad (2)$$

Answer:

Let us again consider the 2D case ($n = 2$) where the equality constraint $\mathbf{Ax} = \mathbf{b}$ resembles p hyperplanes (lines) of the form $\mathbf{a}_i^T \mathbf{x} = b_i$



We will try to solve the more familiar problem

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x}_0 - \mathbf{x}\|_2^2$$

subject to $\mathbf{Ax} = \mathbf{b}$

or more conveniently written

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x}_0 - \mathbf{x}\|_2^2$$

subject to $h_i := \mathbf{a}_i^T \mathbf{x} - b_i = 0, \quad i = 1, \dots, p$

which is equivalent to the initial problem. The constraint set $\mathbf{Ax} = \mathbf{b}$ is convex since for any $\theta \in [0, 1]$ and $\mathbf{x}_1, \mathbf{x}_2$ in the set we get

$$\left. \begin{array}{l} \theta \mathbf{Ax}_1 = \theta \mathbf{b} \\ (1 - \theta) \mathbf{Ax}_2 = (1 - \theta) \mathbf{b} \end{array} \right\} \xRightarrow{(+)} \theta \mathbf{Ax}_1 + (1 - \theta) \mathbf{Ax}_2 = \theta \mathbf{b} + (1 - \theta) \mathbf{b}$$

$$\Rightarrow \mathbf{A}[\theta \mathbf{x}_1 + (1 - \theta) \mathbf{x}_2] = \mathbf{b}$$

$$\Rightarrow \mathbf{A}\hat{\mathbf{x}} = \mathbf{b}$$

Therefore the “new” problem we are about to solve falls under the category of “Projection onto a convex set”.

The gradient of the cost function $\nabla f_0(\mathbf{x}) = \mathbf{x} - \mathbf{x}_0$

The KKT conditions write

$$\begin{cases} \nabla f_0(\mathbf{x}) + \mathbf{A}^T \mathbf{v} = \mathbf{0} \\ h_i(\mathbf{x}) = 0 \end{cases} \Rightarrow \begin{cases} \mathbf{x} - \mathbf{x}_0 + \mathbf{A}^T \mathbf{v} = \mathbf{0} \\ \mathbf{A}\mathbf{x} = \mathbf{b} \end{cases} \Rightarrow \begin{cases} \mathbf{x} + \mathbf{A}^T \mathbf{v} = \mathbf{x}_0 \\ \mathbf{A}\mathbf{x} + \mathbf{O} = \mathbf{b} \end{cases}$$

The above system can easily be written in matrix form as

$$\begin{bmatrix} \mathbb{I}_{n \times n} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{O} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{b} \end{bmatrix}$$

Matrix $\mathbf{M} := \begin{bmatrix} \mathbb{I}_{n \times n} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{O} \end{bmatrix}$ is of $(n+p) \times (n+p)$ dimensions and is in fact invertible (non-zero determinant). Therefore, the above system of equations can be solved as

$$\begin{bmatrix} \mathbf{x}_* \\ \mathbf{v}_* \end{bmatrix} = \mathbf{M}^{-1} \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{b} \end{bmatrix} = \mathbf{z} \in \mathbb{R}^{n+p}$$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$\boxed{(\mathbf{x}_*, \boldsymbol{\lambda}_*) = (\mathbf{z}(1:n, :), \mathbf{z}(n+1:p, :))}$$

4. (20) Let $\mathbb{S} := \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x}\|_2 \leq 1, x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\}$. Solve the problem

$$\min_{\mathbf{x} \in \mathbb{S}} f_0(\mathbf{x}), \tag{3}$$

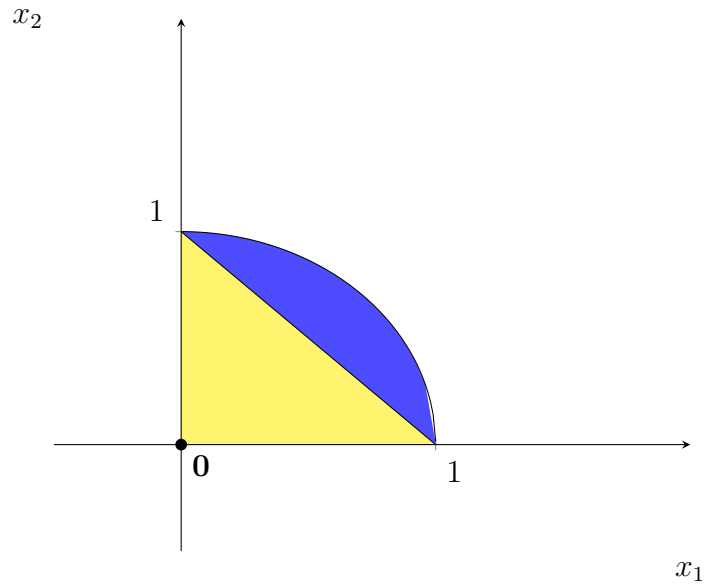
with

(a) $f_0(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 2)^2,$

(b) $f_0(\mathbf{x}) = (x_1 + 2)^2 + (x_2 + 2)^2.$

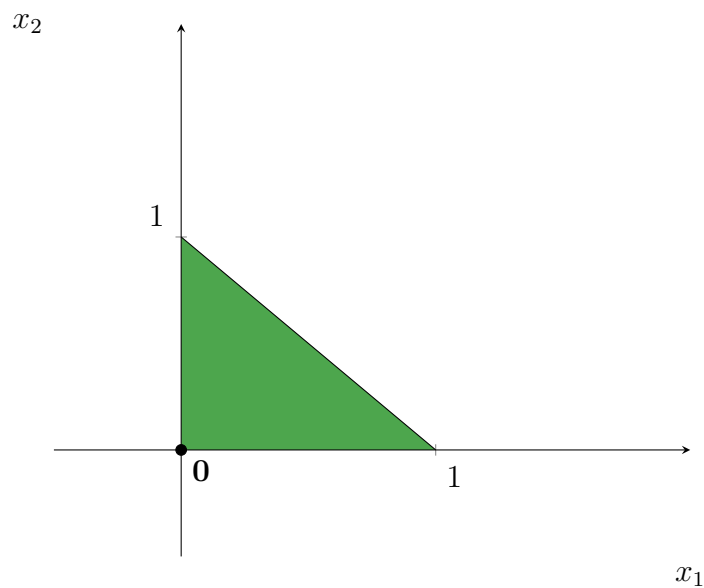
Draw a picture of $\mathbb{S}$ and check if there are any redundant constraints. Then, derive and solve the KKTs.

Answer :



We can easily see that the constraint $\|\mathbf{x}\| \leq 1$ can be removed without changing anything in the structure of our problem, since its intersection with all the other constraints gives the same feasible set. That is, the intersection of the yellow and the blue area

gives the final reduced set $\mathbb{S} = \{\mathbf{x} \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\}$ to be



The problem in a more formal format writes

$$\begin{aligned} & \min_{\mathbf{x}} f_0(\mathbf{x}) \\ & \text{subject to } f_1(\mathbf{x}) := -x_1 \leq 0 \\ & \quad f_2(\mathbf{x}) := -x_2 \leq 0 \\ & \quad f_3(\mathbf{x}) := x_1 + x_2 - 1 \leq 0 \end{aligned}$$

The gradients of the inequality constraints are

$$\nabla f_1(\mathbf{x}) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \nabla f_2(\mathbf{x}) = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \text{ and } \nabla f_3(\mathbf{x}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

(a) The gradient of the cost function is

$$\nabla f_0(\mathbf{x}) = \begin{bmatrix} 2(x_1 - 2) \\ 2(x_2 - 2) \end{bmatrix}$$

So, the KKTs are

$$\begin{aligned} & \begin{cases} \nabla f_0(\mathbf{x}) + \sum_{i=1}^3 \lambda_i \nabla f_i(\mathbf{x}) = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_i f_i(\mathbf{x}) = 0 \end{cases} \\ \Rightarrow & \begin{cases} 2 \begin{bmatrix} x_1 - 2 \\ x_2 - 2 \end{bmatrix} + \lambda_1 \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} 0 \\ -1 \end{bmatrix} + \lambda_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \mathbf{0} \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ -\lambda_1 x_1 = 0, -\lambda_2 x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{cases} \\ \Rightarrow & \begin{cases} 2x_1 - 4 - \lambda_1 + \lambda_3 = 0 \\ 2x_2 - 4 - \lambda_2 + \lambda_3 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{cases} \end{aligned}$$

• If $\lambda_3 = 0$

$$\begin{cases} 2x_1 - 4 - \lambda_1 = 0 \\ 2x_2 - 4 - \lambda_2 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0 \end{cases}$$

$$\Rightarrow \begin{cases} \lambda_1 = 2x_1 - 4 \\ \lambda_2 = 2x_2 - 4 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ x_1(2x_1 - 4) = 0, x_2(2x_2 - 4) = 0 \end{cases}$$

$$\Rightarrow \begin{cases} \lambda_1 = 2x_1 - 4 \\ \lambda_2 = 2x_2 - 4 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ x_1 = 0 \text{ OR } x_1 = 2, x_2 = 0 \text{ OR } x_2 = 2 \end{cases}$$

We notice that, if either one of $x_1, x_2 = 0$ then both $\lambda_1, \lambda_2 = -4$, which is contradictory to the constraint $\lambda_i \geq 0$.

So, the only possible case is $x_1 = x_2 = 2$, which means that $\lambda_1, \lambda_2 = 0$.

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$(\mathbf{x}_*, \boldsymbol{\lambda}_*) = \left(\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right)$$

- If $\lambda_3 > 0 \Rightarrow f(\mathbf{x}) = 0 \Rightarrow x_1 + x_2 = 1$

Multiplying the first two equations by λ_1, λ_2 we get

$$\begin{cases} 2\lambda_1 x_1 - 4\lambda_1 - \lambda_1^2 + \lambda_3 \lambda_1 = 0 \\ 2\lambda_2 x_2 - 4\lambda_2 - \lambda_2^2 + \lambda_3 \lambda_2 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{cases}$$

From the last two equations, some terms get erased giving us

$$\begin{cases} -4\lambda_1 - \lambda_1^2 + \lambda_3 \lambda_1 = 0 \\ -4\lambda_2 - \lambda_2^2 + \lambda_3 \lambda_2 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{cases}$$

$$\Rightarrow \begin{cases} \lambda_1^2 + \lambda_1(4 - \lambda_3) = 0 \\ \lambda_2^2 + \lambda_2(4 - \lambda_3) = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{cases}$$

From the quadratic formula we get for both λ_1, λ_2

$$\Delta = (4 - \lambda_3)^2$$

$$\frac{-(4 - \lambda_3) \pm \sqrt{(4 - \lambda_3)^2}}{2} = \frac{-(4 - \lambda_3) \pm (4 - \lambda_3)}{2} = \begin{cases} -4 - \lambda_3 < 0 \\ \frac{-4 - \lambda_3 + 4 + \lambda_3}{2} = 0 \end{cases}$$

So, $\lambda_1, \lambda_2 = 0$ and from the initial two equations

$$\begin{cases} \lambda_3 = -2x_1 + 4 \\ \lambda_3 = -2x_2 + 4 \end{cases}$$

Which means that $x_1 = x_2$, which when substituting in the last equation gives

$$\lambda_3(x_1 + x_1 - 1) = 0 \Rightarrow \lambda_3(2x_1 - 1) = 0 \xRightarrow{\lambda_3 > 0} 2x_1 - 1 = 0 \Rightarrow x_1 = x_2 = 1/2$$

giving us $\lambda_3 = -2\frac{1}{2} + 4 = 3$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$(\mathbf{x}_*, \boldsymbol{\lambda}_*) = \left(\begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix} \right)$$

(b) The gradient of the cost function is $\nabla f_0(\mathbf{x}) = \begin{bmatrix} 2(x_1 + 2) \\ 2(x_2 + 2) \end{bmatrix}$

So, the KKTs are

$$\begin{aligned} & \begin{cases} \nabla f_0(\mathbf{x}) + \sum_{i=1}^3 \lambda_i \nabla f_i(\mathbf{x}) = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_i f_i(\mathbf{x}) = 0 \end{cases} \\ \Rightarrow & \begin{cases} 2 \begin{bmatrix} x_1 + 2 \\ x_2 + 2 \end{bmatrix} + \lambda_1 \begin{bmatrix} -1 \\ 0 \end{bmatrix} + \lambda_2 \begin{bmatrix} 0 \\ -1 \end{bmatrix} + \lambda_3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \mathbf{0} \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ -\lambda_1 x_1 = 0, -\lambda_2 x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{cases} \\ \Rightarrow & \begin{cases} 2x_1 + 4 - \lambda_1 + \lambda_3 = 0 \\ 2x_2 + 4 - \lambda_2 + \lambda_3 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{cases} \end{aligned}$$

• If $\lambda_3 = 0$

$$\begin{cases} \lambda_1 = 2x_1 + 4 \\ \lambda_2 = 2x_2 + 4 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{cases}$$

$$\Rightarrow \left\{ \begin{array}{l} \lambda_1 = 2x_1 + 4 \\ \lambda_2 = 2x_2 + 4 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ (2x_1 + 4)x_1 = 0, (2x_2 + 4)x_2 = 0, \lambda_3(x_1 + x_2 - 1) = 0 \end{array} \right.$$

$$\Rightarrow \left\{ \begin{array}{l} \lambda_1 = 2x_1 + 4 \\ \lambda_2 = 2x_2 + 4 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ x_1 = 0 \text{ OR } x_1 = -2 < 0, x_2 = 0 \text{ OR } x_2 = -2 < 0 \\ \lambda_3(x_1 + x_2 - 1) = 0 \end{array} \right.$$

Since $x_1, x_2 \geq 0$ the only possible value they can attain is 0, giving us from the first two equations $\lambda_1, \lambda_2 = 4$

Thus the optimal pair of $(\mathbf{x}, \lambda)$ would be

$$(\mathbf{x}_*, \boldsymbol{\lambda}_*) = \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 4 \\ 4 \\ 0 \end{bmatrix} \right)$$

- If $\lambda_3 > 0 \Rightarrow f_3(\mathbf{x}) = 0 \Rightarrow x_1 + x_2 = 1$

Multiplying the first two equations by λ_1, λ_2 we get

$$\left\{ \begin{array}{l} 2\lambda_1 x_1 + 4\lambda_1 - \lambda_1^2 + \lambda_3 \lambda_1 = 0 \\ 2\lambda_2 x_2 + 4\lambda_2 - \lambda_2^2 + \lambda_3 \lambda_2 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{array} \right.$$

From the last two equations, some terms get erased giving us

$$\left\{ \begin{array}{l} 4\lambda_1 - \lambda_1^2 + \lambda_3 \lambda_1 = 0 \\ 4\lambda_2 - \lambda_2^2 + \lambda_3 \lambda_2 = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{array} \right.$$

$$\Rightarrow \left\{ \begin{array}{l} \lambda_1^2 - \lambda_1(4 + \lambda_3) = 0 \\ \lambda_2^2 - \lambda_2(4 + \lambda_3) = 0 \\ \lambda_i \geq 0, f_i(\mathbf{x}) \leq 0 \\ \lambda_1 x_1 = 0, \lambda_2 x_2 = 0, x_1 + x_2 - 1 = 0 \end{array} \right.$$

From the quadratic formula we get for both λ_1, λ_2

$$\Delta = [-(4 + \lambda_3)]^2 = (4 + \lambda_3)^2$$

$$\frac{-[-(4 + \lambda_3)] \pm \sqrt{(4 + \lambda_3)^2}}{2} = \frac{(4 + \lambda_3) \pm (4 + \lambda_3)}{2} = \left\{ \begin{array}{l} 4 + \lambda_3 \\ \frac{4 + \lambda_3 - 4 - \lambda_3}{2} = 0 \end{array} \right.$$

We can recognize the following cases regarding the values of λ_1, λ_2 , which are also symmetric when it comes to the values of the optimal pairs they derive.

– If $\lambda_1 = 0, \lambda_2 = 4 + \lambda_3$, from the initial top 2 equations we derive

$$\begin{cases} \lambda_3 = 2x_1 + 4 \\ 2x_2 + 4 - 4 - \lambda_3 + \lambda_3 = 0 \end{cases} \Rightarrow \begin{cases} x_1 = \frac{\lambda_3 - 4}{2} \\ x_2 = 0 \end{cases}$$

Thus from the equation $x_1 + x_2 = 1 \Rightarrow x_1 = 1 \Rightarrow \frac{\lambda_3 - 4}{2} = 1 \Rightarrow \lambda_3 = 6$ and $\lambda_1 = 10$.

By applying the same logic for the case where $\lambda_1 = 4 + \lambda_3, \lambda_2 = 0$ we get $x_1 = 0, x_2 = 1$ and $\lambda_1 = 10, \lambda_2 = 0, \lambda_3 = 10$.

– If $\lambda_1 = 0, \lambda_2 = 0$ from the initial two equations

$$\begin{cases} \lambda_3 = -2x_1 - 4 \\ \lambda_3 = -2x_2 - 4 \end{cases}$$

Which means that $x_1 = x_2$, which when substituting in the last equation gives

$$\lambda_3(x_1 + x_1 - 1) = 0 \Rightarrow \lambda_3(2x_1 - 1) = 0 \xrightarrow{\lambda_3 > 0} 2x_1 - 1 = 0 \Rightarrow x_1 = x_2 = 1/2$$

giving us $\lambda_3 = -2\frac{1}{2} - 4 = -5 < 0$.

Therefore, this case does not give us an optimal pair $(\mathbf{x}, \boldsymbol{\lambda})$ that satisfies all the constraints.

– If $\lambda_1 = 4 + \lambda_3, \lambda_2 = 4 + \lambda_3$ from the initial two equations

$$\begin{cases} 2x_1 + 4 + \lambda_3 - 4 - \lambda_3 = 0 \\ 2x_2 + 4 + \lambda_3 - 4 - \lambda_3 = 0 \end{cases}$$

Which means that $x_1 = x_2 = 0$, which when substituting in the last equation gives

$$\lambda_3(0 + 0 - 1) = 0 \Rightarrow \lambda_3 = 0 \Rightarrow \lambda_1 = \lambda_2 = 4.$$

All in all, the optimal pairs of $(\mathbf{x}, \boldsymbol{\lambda})$ can be found aggregated in the following table:

λ_1/λ_2	0	$4 + \lambda_3$
0	—	$\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 10 \\ 6 \end{bmatrix} \right)$
$4 + \lambda_3$	$\left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 10 \\ 0 \\ 6 \end{bmatrix} \right)$	$\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 4 \\ 4 \\ 0 \end{bmatrix} \right)$